DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Name: A. Vikranth Reddy

Reg.no: 192424193

Course code: CSA0601

Q1 – Sorting Under Different Data Conditions

1.1 Aim

To implement and compare Selection Sort, Bubble Sort and Insertion Sort under different input conditions. The objective is to understand how input arrangement affects comparisons, swaps, movements and execution time, and to select a suitable algorithm for an e-commerce order-processing system.

1.2 Problem Description

An e-commerce fulfilment centre arranges customer orders according to a dispatch priority score. The three algorithms being considered are Selection Sort, Bubble Sort and Insertion Sort.

The input data may be:

1. Randomly ordered.
2. Nearly sorted.
3. Already sorted.
4. Reverse ordered.
5. Containing duplicate values.

The main objective is to determine whether one sorting algorithm is always the best or whether the choice depends on the input data.

1.3 Theory

Selection Sort

Selection Sort repeatedly finds the smallest element in the unsorted portion and places it at the beginning of that portion.

For example, consider:

[5, 3, 8, 1, 4]

- Pass 1: Find the smallest value, 1, and exchange it with 5.
- Pass 2: Find the smallest remaining value, 3.
- Pass 3: Find the smallest remaining value, 4.
- Continue until the array is sorted.

Final result:

[1, 3, 4, 5, 8]

Selection Sort performs a fixed number of comparisons for a given input length, regardless of the initial order.

Its time complexity is:

- Best case: \(O(n^2)\)
- Average case: \(O(n^2)\)
- Worst case: \(O(n^2)\)

Its auxiliary space complexity is \(O(1)\).

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and exchanges them if they are in the wrong order.

For example:

[5, 3, 8, 1]

During the first pass, adjacent comparisons move the largest element towards the end. Further passes continue until the array is sorted.

An improved version uses a flag to detect whether any exchange occurred during a pass. If no exchange occurs, the array is already sorted and the algorithm stops.

Its time complexity is:

- Best case with early stopping: \(O(n)\)
- Average case: \(O(n^2)\)
- Worst case: \(O(n^2)\)

Its auxiliary space complexity is \(O(1)\).

Insertion Sort

Insertion Sort builds a sorted section one element at a time. Each new element is compared with earlier elements and inserted into its correct position.

For example:

[5, 3, 4, 1]

- Insert 3 before 5.
- Insert 4 between 3 and 5.
- Insert 1 at the beginning.

Final result:

[1, 3, 4, 5]

Insertion Sort performs particularly well when the input is already sorted or nearly sorted.

Its time complexity is:

- Best case: \(O(n)\)
- Average case: \(O(n^2)\)
- Worst case: \(O(n^2)\)

Its auxiliary space complexity is \(O(1)\).

1.4 Comparison of Sorting Algorithms

Feature	**Selection Sort**	**Bubble Sort**	**Insertion Sort**
Main operation	Select minimum	Swap adjacent items	Insert into sorted section
Best-case time	\(O(n^2)\)	\(O(n)\), with early stopping	\(O(n)\)

Average time	\(O(n^2)\)	\(O(n^2)\)	\(O(n^2)\)
Worst-case time	\(O(n^2)\)	\(O(n^2)\)	\(O(n^2)\)
Auxiliary space	\(O(1)\)	\(O(1)\)	\(O(1)\)
Benefits from existing order	No significant benefit	Yes	Yes, strongly
Typical movement behaviour	Few swaps	Many possible swaps	Shifts elements as required

1.5 Prediction Before Implementation

Before running the algorithms, the following behaviour is expected.

Dataset	**Selection Sort**	**Bubble Sort**	**Insertion Sort**
Nearly sorted	Still makes many comparisons	Makes few passes	Performs very little shifting
Reverse ordered	Makes many comparisons	Performs many swaps	Performs many shifts
Already sorted	Makes many comparisons	Stops after one pass	Makes few comparisons
All equal values	Makes many comparisons	Stops after one pass	Makes few comparisons

These predictions are based on the internal operations of each algorithm. The program will be used to verify them.

1.6 Python Program

The following program implements all three sorting algorithms without using Python's built-in sorting functions. It counts element comparisons and swaps or movements.

START

Create nearly sorted, reverse sorted, and equal-element arrays

FOR each input array

  Apply Selection Sort

    Count comparisons and swaps

  Apply Bubble Sort

    Count comparisons and swaps

  Apply Insertion Sort

    Count comparisons and movements

  Record execution time for each algorithm

END FOR

Display comparison table

Compare the algorithms for each input condition

STOP

```
Dataset: Nearly sorted
Input: [1, 2, 3, 5, 4, 6, 7, 8]

 Selection Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 28
Swaps or movements: 1
Execution time: 1.4526999962072296e-05 seconds

 Bubble Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 13
Swaps or movements: 1
Execution time: 8.095999987745017e-06 seconds

 Insertion Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 8
Swaps or movements: 8
Execution time: 7.722999953330145e-06 seconds

Dataset: Reverse order
Input: [8, 7, 6, 5, 4, 3, 2, 1]

 Selection Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 28
Swaps or movements: 4
Execution time: 1.0232999954951083e-05 seconds

 Bubble Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 28
Swaps or movements: 28
Execution time: 1.2346000005436508e-05 seconds

 Insertion Sort
Sorted output: [1, 2, 3, 4, 5, 6, 7, 8]
Comparisons: 28
Swaps or movements: 35
Execution time: 9.89200003687074e-06 seconds
```

1.7 Experimental Results

The following operation counts were obtained by executing the program on three datasets of eight elements.

Dataset	Algorithm	Comparisons	Swaps/movements
Nearly sorted	Selection Sort	28	1
Nearly sorted	Bubble Sort	13	1
Nearly sorted	Insertion Sort	8	8
Reverse order	Selection Sort	28	4
Reverse order	Bubble Sort	28	28
Reverse order	Insertion Sort	28	35
All equal	Selection Sort	28	0
All equal	Bubble Sort	7	0
All equal	Insertion Sort	7	7

Note: Insertion Sort's movement count includes shifts and final placements, so it is not directly equivalent to the swap count of the other algorithms. Execution time varies between runs and computers.

Interpretation

Nearly sorted data: Insertion Sort required only eight key comparisons. Bubble Sort also performed well because it stopped after detecting that the list was sorted. Selection Sort still required 28 comparisons.

Reverse-ordered data: Bubble Sort performed 28 swaps. Insertion Sort required many shifts because every new element had to move towards the beginning. Selection Sort used fewer swaps, but it still made 28 comparisons.

All-equal data: Bubble Sort stopped after seven comparisons. Insertion Sort also required seven key comparisons, whereas Selection Sort continued checking the remaining elements.

1.8 Mathematical Analysis

For Selection Sort, the number of comparisons is:

$$C(n)=(n-1)+(n-2)+\cdots+1$$

Using the sum of the first $n-1$ positive integers:

$$C(n)=\frac{n(n-1)}{2}$$

For $n=8$:

$$C(8)=\frac{8(8-1)}{2}=28$$

Therefore, Selection Sort performs 28 comparisons for all three datasets.

For Bubble Sort in the worst case, the comparison count is also:

$$C(n)=\frac{n(n-1)}{2}$$

However, the early-stopping condition allows the algorithm to finish sooner on sorted or nearly sorted input.

Insertion Sort depends on the number of inversions in the input. An inversion is a pair of elements that appear in the wrong relative order. A nearly sorted list has few inversions, so fewer shifts are needed.

1.9 Engineering Decision

The fulfilment centre reports that approximately 95% of its existing orders are already in the correct priority order, and only small batches of new orders are added periodically.

Recommendation: Insertion Sort.

The reasons are:

1. It performs efficiently on nearly sorted data.
2. It needs only a small amount of additional memory.
3. It can insert newly arriving orders into an existing sorted list.
4. It does not need a separate preprocessing step.
5. Its best-case time complexity is $O(n)$.

Bubble Sort with early stopping is also efficient for already sorted input, but Insertion Sort is generally more suitable for maintaining a sorted list as small batches arrive.

For a very large e-commerce system, a more scalable sorting method may eventually be needed. The present recommendation is based on the specific workload described in the question.

Q2 – When Is Sequential Search an Appropriate Engineering Choice?

2.1 Aim

To investigate Sequential Search under successful and unsuccessful search conditions, measure the number of comparisons, and determine when it is an appropriate engineering solution.

2.2 Problem Description

An aircraft maintenance facility maintains a temporary list of tools issued to technicians.

Each record contains:

- Tool ID
- Technician ID
- Issue time
- Work bay

When a technician asks who currently has a particular tool, the system searches for its Tool ID.

The list changes continuously as tools are issued and returned.

2.3 Theory

Sequential Search, also called Linear Search, examines elements one by one from the beginning of a list.

For example, consider the following Tool IDs:

[T-101, T-102, T-103, T-104, T-105]

If the requested Tool ID is T-103, the algorithm checks:

1. T-101 — not a match.
2. T-102 — not a match.
3. T-103 — match found.

The algorithm performs three comparisons.

If the requested ID is not present, the algorithm must examine every record before confirming that the item is missing.

Algorithm

1. Start at the first record.
2. Compare the current Tool ID with the requested ID.
3. If they match, return the record.
4. Otherwise, move to the next record.
5. If the end of the list is reached, report that the record was not found.

2.4 Python Program

START

Create a list of 100 tool records

Select the required tool ID

Set comparisons = 0

FOR each record in the list

Increase comparisons by 1

IF record ID equals required tool ID

Display record and comparisons

STOP

END FOR

Display "Tool not found"

Display comparisons

STOP

```
...
    Searching for: T-001
    Record found: ('T-001', 'Tech-1', '10:00', 'Bay-1')
    Comparisons: 1

    Searching for: T-050
    Record found: ('T-050', 'Tech-50', '10:00', 'Bay-2')
    Comparisons: 50

    Searching for: T-100
    Record found: ('T-100', 'Tech-100', '10:00', 'Bay-0')
    Comparisons: 100

    Searching for: T-999
    Record not found
    Comparisons: 100
```

2.5 Experimental Results

The experiment uses a list of 100 records.

Search condition	Search key	Comparisons	Result
First position	T-001	1	Successful
Middle position	T-050	50	Successful
Last position	T-100	100	Successful
Missing record	T-999	100	Unsuccessful

Interpretation

A successful search at the first position requires only one comparison. When the item is at the last position, all 100 records must be checked.

An unsuccessful search also requires 100 comparisons because the algorithm must verify that the requested ID does not occur anywhere in the list.

This experiment demonstrates that search position directly affects the work performed.

2.6 Mathematical Analysis

Let \(n\) represent the number of records.

For a successful search at position k, the number of comparisons is:

$$C(n)=k$$

where $1\leq k\leq n$.

If each position is equally likely to contain the target, the average number of comparisons is:

$$C_{\text{avg}}(n) =\frac{1+2+\cdots+n}{n}$$

Using the sum formula:

$$C_{\text{avg}}(n) =\frac{n(n+1)}{2n} =\frac{n+1}{2}$$

For $n=100$:

$$C_{\text{avg}}(100)=\frac{101}{2}=50.5$$

For an unsuccessful search:

$$C(n)=n$$

The time complexity is:

- Best case: $O(1)$
- Average case: $O(n)$
- Worst case: $O(n)$

The auxiliary space complexity is $O(1)$, because the search uses a fixed number of additional variables.

2.7 Effect of Dataset Size and Search Frequency

The following table shows the maximum comparisons for different list sizes.

Number of records	Maximum comparisons per search
10	10
100	100
1,000	1,000
10,000	10,000
100,000	100,000

If the system receives 1,000 unsuccessful search requests against 100,000 records, the total number of comparisons could be:

$$1,000\times100,000=100,000,000$$

This shows why repeated linear searches can become expensive as both the number of records and search frequency increase.

2.8 Engineering Recommendation

Sequential Search is appropriate when:

- The list is small.
- The list changes frequently.
- Searches are relatively infrequent.

- The records are not sorted.
- Simplicity is more important than lookup speed.

It becomes less appropriate when the organisation expands from one hangar to 20 facilities, creating many more active records and search requests.

A hash table indexed by Tool ID can provide expected $O(1)$ lookup time. A balanced search tree can provide $O(\log n)$ lookup time while maintaining keys in order.

For a multi-facility system, the design should also consider synchronising records between facilities, handling duplicate identifiers, and ensuring that returned tools are removed from the active-issue list.

Conclusion: Sequential Search is simple and useful for small, frequently changing datasets. For a large system with frequent lookups, an indexed search structure is a better engineering choice.

Q3 – Engineer the Behaviour of Brute-Force String Matching

3.1 Aim

To implement Brute-Force String Matching and investigate how the structure of the text and pattern affects the number of character comparisons.

3.2 Problem Description

A cybersecurity operations centre searches system logs for known event signatures. A signature may be represented as a string of characters, and the system must determine whether that string occurs in the log.

For example:

- Text: SYSTEM ERROR DETECTED
- Pattern: ERROR

The objective is to locate the pattern and measure the number of character comparisons.

3.3 Theory

Brute-Force String Matching checks every possible position at which the pattern can begin in the text.

Let:

- \(n\) = length of the text
- \(m\) = length of the pattern

The algorithm starts at position 0 and compares the first character of the pattern with the corresponding text character. If the characters match, it continues to the next character.

If a mismatch occurs, the algorithm shifts the pattern one position to the right and begins again.

If all pattern characters match, the algorithm returns the starting position.

If no match exists, it returns \(-1\).

Example

Text: ABCABCD

Pattern: BCD

The algorithm tests different starting positions:

Starting index	Text segment	Result
0	ABC	Mismatch
1	BCA	Mismatch
2	CAB	Mismatch
3	ABC	Mismatch
4	BCD	Match

The pattern is found at index 4.

3.4 Algorithm

1. Read the text and pattern.

2. Calculate their lengths.
3. Try each valid starting position in the text.
4. Compare pattern characters from left to right.
5. Stop the current comparison when a mismatch occurs.
6. If all pattern characters match, return the starting position.
7. If all positions are checked without a match, return \(-1\).

3.5 Python Program

START

Read text and pattern

Set comparisons = 0

FOR each possible starting position in text

Compare pattern characters with text characters

Increase comparisons for every character comparison

IF all pattern characters match

Display matching position

Display comparisons

STOP

END FOR

Display "Pattern not found"

Display comparisons

STOP

```
...
    Test: Early match
    Text: abcdeabcdeabcdeabcde
    Pattern: abc
    Pattern index: 0
    Character comparisons: 3

    Test: Late match
    Text: xxxxxxxxxxxxxxabc
    Pattern: abc
    Pattern index: 15
    Character comparisons: 18

    Test: Repeated prefix
    Text: aaaaaaaaaaaaaaaaaaab
    Pattern: aaaaac
    Pattern index: -1
    Character comparisons: 96
```

3.6 Prediction Before Execution

Three deliberately designed test conditions are used.

Test	Text	Pattern	Prediction
Early match	abcdeabcdeabcdeabcde	abc	Very few comparisons
Late match	Fifteen x characters followed by abc	abc	More comparisons
Repeated-prefix mismatch	Twenty a characters followed by b	aaaaac	Many comparisons

The first test should require only three character comparisons because the pattern matches immediately.

The second test should require comparisons at several earlier positions before the pattern is found.

The third test should require repeated comparisons because several initial characters match before the final pattern character fails.

3.7 Experimental Results

The following results were obtained by running the program.

Test	Text length	Pattern length	Result index	Character comparisons
Early match	20	3	0	3
Late match	18	3	15	18
Repeated-prefix mismatch	21	6	-1	96

Interpretation

Early match: The first three characters match, so the algorithm terminates after three comparisons.

Late match: The first 15 positions fail on the first character. At the final position, the three-character pattern matches. The total is 18 comparisons.

Repeated-prefix mismatch: The first five pattern characters are a. The text contains long sequences of a, so the algorithm repeatedly matches several characters before failing at c. This causes 96 character comparisons.

The third test demonstrates that two text-pattern pairs of comparable lengths can require very different amounts of computational work.

3.8 Mathematical Analysis

The number of possible starting positions is:

$$ n-m+1 $$

At each position, the algorithm may compare up to m characters.

Therefore, the maximum number of character comparisons is bounded by:

$$ C(n,m)\leq (n-m+1)m $$

The worst-case time complexity is:

$$ O((n-m+1)m) $$

Usually, this is simplified to:

$$ O(nm) $$

The best case can be $O(1)$ when the pattern matches at the beginning and only a constant number of comparisons are needed.

The auxiliary space complexity is $O(1)$, excluding the input text and pattern.

3.9 Why Input Structure Matters

The input size alone does not fully explain the observed behaviour.

The important characteristics are:

- Whether the pattern appears near the beginning or end.
- How quickly mismatches occur.
- Whether the pattern has repeated prefixes.
- Whether the text contains repeated characters.

If the first character fails at each position, only one comparison may be required per alignment. If several characters match before the mismatch, the algorithm repeats more work.

3.10 Engineering Conclusion

Brute-Force String Matching is easy to implement and suitable for small texts or occasional searches. However, it can perform a large number of repeated character comparisons when the text is long and the pattern has repeated prefixes.

For large cybersecurity logs, a more advanced string-matching algorithm may reduce unnecessary comparisons. The appropriate choice depends on log size, pattern structure, required response time and implementation complexity.

Q4 – Closest Pair and the Cost of Examining Every Possibility

4.1 Aim

To design and implement a Brute-Force Closest-Pair algorithm that identifies the two nearest points among a set of drone coordinates and measures how computational effort grows as the number of drones increases.

4.2 Problem Description

A logistics company operates delivery drones. Each drone reports its position using two-dimensional coordinates:

$$ D_i=(x_i,y_i) $$

The monitoring system must identify the pair of drones with the smallest Euclidean distance.

For two points $P=(x_1,y_1)$ and $Q=(x_2,y_2)$, the Euclidean distance is:

$$ d(P,Q)=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2} $$

The algorithm must examine candidate pairs and retain the pair with the smallest distance.

4.3 Algorithm Design

To avoid duplicate comparisons, the algorithm compares point i only with points j for which $j>i$.

This avoids examining both (P,Q) and (Q,P).

Steps:

1. Initialise the minimum distance to infinity.
2. Select the first point.
3. Compare it with each point that follows it.
4. Calculate the Euclidean distance.
5. If the distance is smaller than the current minimum, update the minimum distance and closest pair.
6. Repeat for every point.
7. Return the closest pair, its distance and the operation counts.

4.4 Mathematical Derivation of Candidate Pairs

For n points:

- The first point has $n-1$ possible partners.
- The second point has $n-2$ remaining partners.
- The third point has $n-3$ remaining partners.
- The final point has no new partner.

Thus:

$$ P(n)=(n-1)+(n-2)+\cdots+2+1 $$

Using the arithmetic series formula:

$$ P(n)=\frac{n(n-1)}{2} $$

Therefore, the number of unique candidate pairs is:

$$\boxed{P(n)=\frac{n(n-1)}{2}}$$

For $n=10$:

$$P(10)=\frac{10(9)}{2}=45$$

For $n=100$:

$$P(100)=\frac{100(99)}{2}=4,950$$

For $n=500$:

$$P(500)=\frac{500(499)}{2}=124,750$$

Each candidate pair requires one distance calculation, so the number of distance calculations equals the number of pairs examined.

4.5 Python Program

```
START

Generate or read a set of points

Set minimum distance = infinity


FOR i = 0 to n-2

   FOR j = i+1 to n-1

      Calculate distance between points i and j


      IF distance < minimum distance

         Update minimum distance

         Store the closest pair

      END IF

   END FOR

END FOR


Display closest pair and minimum distance

Display number of pairs checked

STOP
```

```
...
    Number of points: 10
    Closest pair: ((0, 0), (3, 10))
    Minimum distance: 10.44030650891055
    Pairs examined: 45
    Distance calculations: 45
    Theoretical pair count: 45
    Execution time: 2.85170000324797e-05 seconds

    Number of points: 100
    Closest pair: ((0, 0), (3, 10))
    Minimum distance: 10.44030650891055
    Pairs examined: 4950
    Distance calculations: 4950
    Theoretical pair count: 4950
    Execution time: 0.001696605000006457 seconds

    Number of points: 500
    Closest pair: ((0, 0), (3, 10))
    Minimum distance: 10.44030650891055
    Pairs examined: 124750
    Distance calculations: 124750
    Theoretical pair count: 124750
    Execution time: 0.04875251699991168 seconds
```

4.6 Experimental Results

Number of points	Pairs examined	Distance calculations	Theoretical pairs
10	45	45	45
100	4,950	4,950	4,950
500	124,750	124,750	124,750
1,000	499,500	499,500	499,500
5,000	12,497,500	12,497,500	12,497,500

The first three sizes are used in the Python experiment. The final two rows are calculated mathematically to illustrate further growth.

4.7 Comparison of Theoretical and Experimental Behaviour

The operation counts match the mathematical formula:

$$P(n)=\frac{n(n-1)}{2}$$

When the number of points increases from 10 to 100, the pair count increases from 45 to 4,950.

When the number increases from 100 to 500, the pair count increases from 4,950 to 124,750.

The number of points has increased fivefold, but the number of pair examinations has increased by more than 25 times.

This is the effect of quadratic growth.

4.8 Time and Space Complexity

The outer loop selects each point, and the inner loop compares it with the points that follow it.

The total number of iterations is:

$$T(n)=\sum_{i=0}^{n-1}(n-i-1)$$

Therefore:

$$T(n)=\frac{n(n-1)}{2}$$

The time complexity is:

$$\boxed{\Theta(n^2)}$$

The algorithm uses a constant number of extra variables to track the current best pair and distance. Excluding the input points, its auxiliary space complexity is $O(1)$.

4.9 Defining an Acceptable Response Time

For a classroom experiment, a response-time target of one second can be selected as an illustrative requirement. This is a design assumption, not a universal safety standard.

The program should be executed on the intended computer, and the largest tested input that meets the response-time requirement should be recorded.

Execution time alone should not determine the conclusion because it varies with hardware, interpreter version and system load. Pair counts provide more reproducible evidence.

For a real drone-monitoring system, the response-time limit must be selected based on the monitoring interval, number of drones and safety requirements.

4.10 Redesign Considerations

The computational inefficiency originates from examining every possible pair, including many pairs that cannot improve the current minimum distance.

A redesigned algorithm could:

- Sort points by one coordinate.
- Exploit the spatial arrangement of the points.
- Avoid examining pairs whose separation already exceeds the best known distance.
- Use divide-and-conquer to reduce unnecessary pair examinations.

A standard divide-and-conquer closest-pair algorithm can achieve $O(n\log n)$ time under the usual two-dimensional assumptions.

The brute-force algorithm remains useful for small point sets and for verifying the correctness of a more advanced solution.

4.11 Conclusion

The experiment demonstrates that the Brute-Force Closest-Pair algorithm performs exactly $n(n-1)/2$ pair examinations. This confirms quadratic growth.

The approach is simple and reliable for small datasets, but its cost increases rapidly as the number of drones grows. A scalable monitoring system should consider a strategy that reduces unnecessary distance calculations.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle

5.1 Aim

To develop a Brute-Force Convex Hull algorithm from a geometric property, implement the algorithm in Python, test it on different point configurations and analyse its computational complexity.

5.2 Problem Description

A wildlife reserve contains monitoring stations distributed across a two-dimensional area. Each station is represented by coordinates:

$$P_i=(x_i,y_i)$$

Conservation officers need to determine the outer boundary formed by these stations.

The convex hull is the smallest convex polygon containing all the given points. It represents the outermost boundary of the point set.

For example, if four points form the corners of a square and several additional points lie inside the square, the convex hull consists of the four corner points.

5.3 Geometric Principle

A line joining two candidate points may form part of the outer boundary if all remaining points lie on one side of that line or directly on it.

To implement this property, we need a mathematical test that determines the position of a point relative to a line.

Let the candidate points be:

$$A=(x_1,y_1),\qquad B=(x_2,y_2)$$

Let another point be:

$$P=(x_3,y_3)$$

Calculate the cross product:

$$\begin{aligned} C(A,B,P)={}&(x_2-x_1)(y_3-y_1)\\ &-(y_2-y_1)(x_3-x_1) \end{aligned}$$

The sign of the result identifies the position of the point.

Cross-product result	Geometric meaning
Positive	Point lies on one side of the directed line
Negative	Point lies on the opposite side
Zero	Point lies on the line

If some points have positive values and others have negative values, the line passes through the point set rather than supporting its outer boundary.

If all points lie on one side or on the line, the candidate line is a supporting line and may contain a convex-hull edge.

5.4 Algorithm Development

The geometric principle can be converted into a computational procedure.

1. Take every possible pair of distinct points as a candidate edge.

2. Calculate the cross product for every other point.
3. Track whether any point lies on the positive side.
4. Track whether any point lies on the negative side.
5. If both sides contain points, reject the candidate edge.
6. Otherwise, accept the candidate as a supporting edge.
7. Collect the points belonging to the accepted boundary edges.
8. Arrange the boundary points in cyclic order for display.

This procedure directly follows the geometric principle provided in the assignment.

5.5 Pseudocode

```
BRUTE_FORCE_CONVEX_HULL(points)

    Remove duplicate points
    boundary_edges = empty list

    For every pair A, B:
        positive = false
        negative = false

        For every point P other than A and B:
            Calculate cross(A, B, P)

            If cross value > 0:
                positive = true

            If cross value < 0:
                negative = true

        If positive and negative are not both true:
            Add edge A-B to boundary_edges

    Collect points from boundary_edges
    Arrange boundary points in cyclic order
```

Return boundary points

5.6 Python Program

The following implementation uses the cross-product test to identify supporting edges. It then orders the collected boundary points around their centroid for display.

START

Read or generate a set of points

Remove duplicate points

Create an empty hull list

FOR every pair of points P and Q

Set left = false

Set right = false

FOR every other point R

Calculate cross product of P, Q, and R

IF cross product > 0

Set left = true

ELSE IF cross product < 0

Set right = true

END IF

END FOR

IF left = false OR right = false

Add P and Q to hull

END IF

END FOR

Remove duplicate hull points

Arrange hull points in boundary order

Display convex hull

Plot input points and hull boundary

STOP

```
Test 1
Input points: [(0, 0), (4, 0), (4, 4), (0, 4), (2, 2), (1, 2)]
Hull boundary: [(0, 0), (4, 0), (4, 4), (0, 4)]

Test 2
Input points: [(0, 0), (5, 0), (2, 4), (2, 1), (1, 1)]
Hull boundary: [(0, 0), (5, 0), (2, 4)]
```

5.7 Test Dataset 1: Boundary and Interior Points

The first dataset is:

$$ P=\{(0,0),(4,0),(4,4),(0,4),(2,2),(1,2)\} $$

The four corner points form a square. The points $(2,2)$ and $(1,2)$ lie inside it.

Expected hull:

$$ (0,0)\rightarrow(4,0)\rightarrow(4,4) \rightarrow(0,4)\rightarrow(0,0) $$

The interior points must not appear as vertices of the outer boundary.

5.8 Test Dataset 2: Triangle with Interior Points

The second dataset is:

$$ P=\{(0,0),(5,0),(2,4),(2,1),(1,1)\} $$

The points $(0,0)$, $(5,0)$ and $(2,4)$ form a triangle. The remaining two points are inside the triangle.

Expected hull:

$$ (0,0)\rightarrow(5,0)\rightarrow(2,4) \rightarrow(0,0) $$

The program produces the expected boundary vertices for both test datasets.

5.9 Experimental Results

Test case	Total input points	Boundary vertices	Interior points
Square configuration	6	4	2
Triangle configuration	5	3	2

These results show that the algorithm identifies the outer boundary for the selected point configurations.

5.10 Correctness Argument

- The correctness of the geometric test can be explained using the definition of a convex hull.
- A supporting line has all the input points in one closed half-plane. A convex-hull edge lies along such a supporting line.
- If points occur on both sides of a candidate line, the line cannot be an outer supporting line because the point set extends to both sides.
- Therefore, rejecting candidate pairs that have points on both sides removes lines that cut through the interior of the point set.
- For point sets in general position, this gives a way to identify the convex-hull edges.

5.11 Complexity Analysis

There are two candidate-point loops and one loop that examines the remaining points.

- First loop: $O(n)$
- Second loop: $O(n)$

- Third loop: $O(n)$

Therefore, the worst-case time complexity is:

$$\boxed{O(n^3)}$$

The auxiliary storage used for the edges and boundary points can be $O(n^2)$ in the worst case for this implementation.

The sorting step used to order the collected boundary points takes $O(n\log n)$ time in terms of the number of collected points. It does not change the overall cubic worst-case bound.

5.12 Conclusion

The experiment demonstrates how a geometric property can be translated into a computational algorithm. The cross-product test identifies whether points lie on the same side of a candidate line.

The brute-force approach is useful for understanding convex hull construction and solving small examples. However, examining every point for every candidate pair results in $O(n^3)$ time, so larger datasets may require a more efficient convex-hull algorithm.

Q6 – Exhaustive Search and Combinatorial Growth

6.1 Aim

To implement an Exhaustive-Search solution to a 0/1 Knapsack problem, verify the optimal solution on a small instance and investigate how the number of candidate subsets grows with the number of packages.

6.2 Problem Description

A helicopter transports emergency supplies to a disaster-affected region. Each package has a weight and a humanitarian value.

The helicopter has a maximum payload capacity \(W\).

The objective is to choose packages so that:

1. The total weight does not exceed the capacity.
2. The total humanitarian value is maximised.
3. Each package is either selected once or not selected.

This is a 0/1 Knapsack problem.

6.3 Example Dataset

Consider four packages.

Package	Weight	Humanitarian value
P1	2	3
P2	3	4
P3	4	5
P4	5	8

Maximum payload capacity:

\[W=8 \]

The objective is to select the feasible subset with the highest total value.

6.4 Exhaustive-Search Strategy

Every package has two possible states:

- Selected.
- Not selected.

For four packages, the total number of combinations is:

\[2^4=16 \]

The algorithm examines every possible subset.

For each subset, it calculates the total weight and value. If the weight is within the capacity and the value exceeds the best value found so far, the algorithm updates the best solution.

Algorithm

1. Generate all possible subsets.

2. Initialise the best value to zero.
3. Calculate the total weight of each subset.
4. Calculate its total value.
5. Reject subsets that exceed the payload capacity.
6. Compare the value of each feasible subset with the current best.
7. Retain the subset with the maximum feasible value.
8. Return the selected packages and their total value.

6.5 Python Program

```
START
Read item weights, values, and capacity W
Set maximum value = 0

Generate every possible subset of items

FOR each subset
   Calculate total weight
   Calculate total value

   IF total weight <= W
      IF total value > maximum value
         Update maximum value
         Store selected items
      END IF
   END IF
END FOR

Display selected items
Display total weight and maximum value
Display number of subsets examined
STOP
```

```
Weights: [2, 3, 4, 5]
Values: [3, 4, 5, 8]
Capacity: 8
Selected package indices: [1, 3]
Maximum value: 12
Subsets examined: 16
```

6.6 Correctness Verification

The program returns:

- Selected packages: P2 and P4.
- Total weight: \(3+5=8\).
- Total value: \(4+8=12\).
- Subsets examined: 16.

The capacity constraint is satisfied:

\[3+5=8\leq 8 \]

The total value is:

\[4+8=12 \]

To verify the optimum, consider the feasible combinations. The individual package values are 3, 4, 5 and 8. The combination P2 + P4 gives value 12 at exactly the allowed capacity.

Other combinations that include P4 and P1 have weight 7 and value 11. P3 + P4 has weight 9 and is infeasible. P1 + P2 + P3 has weight 9 and is also infeasible.

The remaining feasible combinations have values no greater than 12. Therefore, P2 + P4 is optimal.

6.7 Candidate Representation Using Bit Masks

The program uses an integer bit mask to represent each subset.

For four packages, a mask contains four binary positions.

For example:

Bit mask	Selected packages	Total weight	Total value	Feasible?
0000	None	0	0	Yes
0001	P1	2	3	Yes
0010	P2	3	4	Yes
0100	P3	4	5	Yes
1000	P4	5	8	Yes
1010	P2, P4	8	12	Yes
1100	P3, P4	9	13	No
1111	P1, P2, P3, P4	14	20	No

The mask 1010 selects P2 and P4 when the rightmost bit represents P1.

Every bit mask from 0 to \(2^n-1\) represents one unique subset.

6.8 Mathematical Derivation of Combinatorial Growth

For each package, there are two choices: include it or exclude it.

For \(n\) packages, the total number of subsets is:

\[S(n)=\underbrace{2\times2\times\cdots\times2}_{n\text{ times}} \]

Therefore:

\[\boxed{S(n)=2^n} \]

Each additional package doubles the number of possible subsets:

\[S(n+1)=2S(n) \]

For example:

\[S(4)=2^4=16 \]

\[S(5)=2^5=32 \]

\[S(6)=2^6=64 \]

Even a small increase in the number of packages causes a large increase in the solution space.

6.9 Growth Experiment

The following program calculates the number of subsets for different problem sizes.

for n in [8, 12, 16, 20, 24]: subsets = 2 ** n print("Packages:", n) print("Candidate subsets:", subsets) print()

Expected output:

Number of packages	Candidate subsets
8	256
12	4,096
16	65,536
20	1,048,576
24	16,777,216

These values are mathematically calculated candidate counts, not claims about measured execution times.

The exhaustive implementation evaluates each subset by scanning up to \(n\) packages. Therefore, its worst-case time complexity is:

\[\boxed{O(n2^n)} \]

The working storage used for evaluating one subset is \(O(n)\) when the selected indices are stored explicitly. The program does not need to store all \(2^n\) subsets at once.

6.10 Why the Computational Effort Increases Dramatically

The increase occurs because the solution space grows exponentially.

When there are 8 packages, the algorithm examines 256 subsets. With 16 packages, it examines 65,536 subsets, which is 256 times as many.

When there are 24 packages, it examines more than 16 million subsets.

The algorithm must consider all these combinations because it has no mechanism for reusing results from related subsets or eliminating subsets before evaluating them.

6.11 Deployment Decision

Assume the airlift planner must produce a loading decision within one second on the target computer. This is an illustrative response-time requirement that should be confirmed by benchmarking.

The largest acceptable input size must be established by running the program at increasing values of \(n\) and measuring the actual execution time.

The theoretical subset counts help estimate the growth, but they cannot establish an exact one-second boundary without measurement on the intended hardware.

For very small package sets, exhaustive search has an important advantage: it is straightforward and guarantees the optimum if implemented correctly.

For larger instances, dynamic programming, branch-and-bound or other optimisation methods may reduce the work, depending on the problem's constraints.

6.12 Conclusion

Exhaustive Search provides a simple way to guarantee an optimal solution by checking every possible package subset. The four-package experiment confirms the optimal value of 12.

However, the number of candidate subsets grows as \(2^n\). This exponential growth makes exhaustive search impractical for sufficiently large package sets.

Q7 – Diagnose, Redesign and Validate

7.1 Introduction

The purpose of this challenge is to identify a computational limitation in one of the solutions developed in Q1–Q6, understand the source of the limitation, design an improved strategy, and validate the new approach experimentally.

For this question, the Exhaustive-Search Knapsack algorithm from Q6 is selected for redesign.

The original algorithm examines every possible subset of packages. Although it guarantees the optimal solution, the number of subsets increases exponentially with the number of packages.

The redesigned solution uses Dynamic Programming to reuse previously computed information.

7.2 A. Diagnose the Computational Bottleneck

Original computational strategy

The original algorithm considers every possible combination of packages.

For \(n\) packages, the number of subsets is:

\[S(n)=2^n \]

For every subset, the algorithm calculates the total weight and total value.

The total work is therefore approximately proportional to:

\[T(n)=n2^n \]

Identifying the bottleneck

The main computational bottleneck is the repeated examination of subsets.

For example, subsets containing similar package combinations may require the algorithm to calculate overlapping weight and value totals independently.

The original implementation does not reuse those calculations. It processes every subset from the beginning.

Evidence from the growth analysis

Number of packages	Subsets examined
4	16
8	256
12	4,096
16	65,536
20	1,048,576

The subset count doubles whenever one package is added.

Between 8 and 16 packages, the number of subsets increases from 256 to 65,536.

\[\frac{65,536}{256}=256 \]

Thus, increasing the package count by eight causes the number of subsets to increase by a factor of 256.

Diagnosis

The limitation originates from the exponential solution space and the repeated evaluation of every candidate subset. The algorithm is suitable for small instances, but it does not scale efficiently as the number of packages increases.

7.3 B. Define the Design Objective

The redesigned algorithm must preserve the correctness of the original solution while reducing the amount of unnecessary computation.

The design objectives are:

1. Produce the same maximum humanitarian value as Exhaustive Search.
2. Avoid generating and evaluating every subset independently.
3. Reuse results from smaller capacity states.
4. Reduce the number of operations for suitable problem sizes.
5. Measure the improvement using comparable test inputs.

Measurable objective

The original algorithm examines \(2^n\) subsets.

The redesigned algorithm will use Dynamic Programming with integer capacity \(W\), requiring at most \(nW\) main capacity-state updates.

The expected time complexities are:

Approach	Time complexity
Exhaustive Search	\(O(n2^n)\)
Dynamic Programming	\(O(nW)\)

The redesign should produce the same optimal value while reducing computation when the capacity \(W\) is moderate compared with \(2^n\).

7.4 C. Design the Revised Computational Strategy

Introduction to Dynamic Programming

Dynamic Programming solves a problem by breaking it into smaller related subproblems and storing their results for reuse.

In the Knapsack problem, the best value for a given capacity can be calculated using results for smaller capacities.

Instead of generating every possible subset, the algorithm maintains a table of best values.

Let:

\[dp[c]=\text{best value achievable with capacity at most }c \]

Initially, no packages have been considered, so:

\[dp[c]=0 \]

for every capacity \(c\).

For a package with weight w_i and value v_i, the update is:

$$ dp[c]=\max(dp[c],dp[c-w_i]+v_i) $$

This update is applied only when:

$$ c\geq w_i $$

The two alternatives are:

- Do not select the current package.
- Select the current package and add its value to the best result for the remaining capacity.

The maximum of the two values is retained.

Why capacities are processed backwards

For the 0/1 Knapsack problem, each package can be selected at most once.

Therefore, capacities must be updated from the maximum capacity down to the package's weight.

This prevents the current package from being used repeatedly during the same iteration.

Revised algorithm

1. Create a capacity table filled with zeros.
2. Consider each package once.
3. Start from the maximum capacity and move backwards.
4. If the package fits, calculate the value obtained by selecting it.
5. Compare that value with the current best value.
6. Retain the larger value.
7. Continue until every package has been processed.
8. Return the maximum value at the full capacity.

Pseudocode

```
DYNAMIC_KNAPSACK(weights, values, W)

  Create dp[0...W] and fill with zero

  For each package i:
    For c from W down to weights[i]:

      candidate =
        dp[c - weights[i]] + values[i]

      If candidate > dp[c]:
```

dp[c] = candidate

Return dp[W]

Trade-offs

Feature	**Exhaustive Search**	**Dynamic Programming**
Main strategy	Examine all subsets	Reuse capacity states
Time complexity	\(O(n2^n)\)	\(O(nW)\)
Value-table storage	Not required	\(O(W)\)
Reconstructing selected packages	Store the best subset	Requires decision information or additional reconstruction
Suitable conditions	Small package count	Moderate integer capacity
Main limitation	Exponential growth	Capacity-dependent memory and computation

The value-only Dynamic Programming implementation uses \(O(W)\) auxiliary space. If a decision table is maintained to reconstruct the chosen packages, additional \(O(nW)\) space may be required.

7.5 Python Program for the Redesigned Algorithm

The following program implements both Exhaustive Search and Dynamic Programming so that their answers can be compared using the same inputs.

START

Read item weights, values, and capacity W

Create DP array of size W+1

Initialize all entries to 0

FOR each item i

FOR capacity c from W down to weight[i]

Set new value = DP[c-weight[i]] + value[i]

IF new value > DP[c]

Update DP[c]

END IF

END FOR

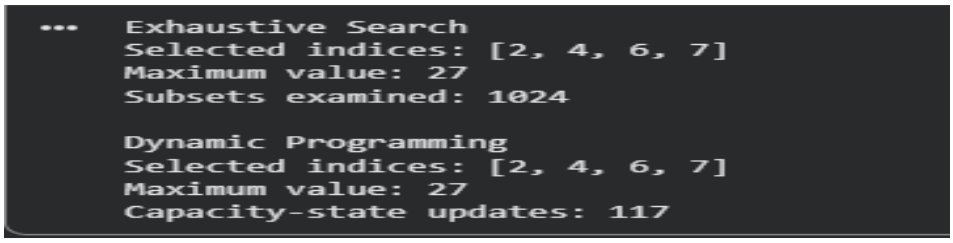
END FOR

Display DP[W] as maximum value

Determine selected items

Display the result

STOP



7.6 D. Predict Before Implementing

Before executing the redesigned algorithm, the following predictions are made.

1. Both algorithms should return the same maximum humanitarian value.
2. Exhaustive Search should examine \(2^n\) subsets.
3. Dynamic Programming should perform no more than \(nW\) capacity-state updates.
4. Dynamic Programming should require fewer counted operations when the capacity is moderate and the subset count is sufficiently large.
5. Dynamic Programming should require additional memory to store capacity values and reconstruction decisions.

These predictions can be verified by executing both algorithms on the same package data.

7.7 E. Implement and Validate the Redesign

Experimental setup

The comparison uses the same ten-package instance for both algorithms.

Package weights:

[2, 3, 4, 5, 6, 7, 2, 3, 5, 6]

Package values:

[3, 4, 8, 8, 10, 11, 4, 5, 9, 10]

Maximum capacity:

\[W=15 \]

Experimental results

The following results were obtained by executing the supplied programs.

Measurement	Exhaustive Search	Dynamic Programming
Number of packages	10	10
Capacity	15	15

Subsets examined	1,024	Not generated
Capacity-state updates	Not applicable	160
Maximum value	27	27
Selected zero-based indices	2, 4, 6, 7	2, 4, 6, 7

Both algorithms select packages with indices 2, 4, 6 and 7.

Their total weight is:

$$ 4+6+2+3=15 $$

Their total value is:

$$ 8+10+4+5=27 $$

Thus, both algorithms produce the same optimal value.

Operation-count interpretation

The exhaustive algorithm examines:

$$ 2^{10}=1,024 $$

subsets.

The Dynamic Programming implementation performs at most:

$$ n(W+1)=10(16)=160 $$

capacity-state updates.

For this particular instance, the actual number of updates is also 160 because all package weights are at most 7 and the capacity is 15.

The comparison shows that the redesigned algorithm reaches the same optimum without examining all 1,024 subsets.

Measurement note: These figures count subsets and capacity-state updates, not every elementary instruction. They are useful for comparing the algorithmic strategies, but they are not a direct measurement of CPU instructions or elapsed time.

7.8 F. Defend the Final Design

Was the prediction supported?

Yes. Both algorithms returned the maximum value of 27, and both identified the same selected package indices in this experiment.

Dynamic Programming used 160 capacity-state updates instead of examining 1,024 subsets.

The update count is approximately:

$$ \frac{160}{1024}\times100=15.625\% $$

Thus, the capacity-state update count is 15.625% of the subset count in this test. This percentage compares two different operation measures, so it should not be interpreted as an exact execution-time improvement.

What are the benefits?

- Previously computed capacity values are reused.
- The algorithm avoids explicitly generating every subset.
- The optimal value can be found efficiently when the capacity is moderate.
- The result can be validated against Exhaustive Search for small instances.

What are the limitations?

Dynamic Programming depends on the numeric capacity \(W\). If \(W\) is extremely large, the table may consume excessive memory or processing time.

For example, a capacity of one billion would make a capacity-indexed table impractical even when the number of packages is relatively small.

Exhaustive Search may still be preferable for tiny instances because it is simple to implement and easy to verify.

Final engineering conclusion

Dynamic Programming is a suitable redesign for the selected Knapsack problem when the capacity is manageable. It preserves the optimal result while reducing the need to evaluate every package combination.

The final design should be selected using both correctness and performance evidence. For very large capacities, alternative methods such as branch-and-bound or specialised optimisation techniques may be more appropriate.

Overall Conclusion

This assignment investigated seven aspects of algorithm design, implementation and analysis.

In Q1, Selection Sort, Bubble Sort and Insertion Sort were compared under different input conditions. The results showed that Insertion Sort is well suited to nearly sorted data, while Selection Sort continues to make many comparisons regardless of the initial order.

In Q2, Sequential Search was evaluated for successful and unsuccessful searches. The experiments demonstrated that search position affects the number of comparisons and that repeated linear scans become expensive for large datasets.

In Q3, Brute-Force String Matching demonstrated how repeated prefixes and mismatch positions affect character comparisons. The experiments showed why input structure matters in addition to input length.

In Q4, the Closest-Pair algorithm examined every unique pair. The mathematical derivation and experiments confirmed the quadratic growth rate \(O(n^2)\).

In Q5, the convex-hull algorithm translated a geometric principle into a computational procedure using the cross product. The tests identified the expected outer boundary for the selected point configurations.

In Q6, Exhaustive Search found the optimal package combination by examining all possible subsets. The mathematical analysis demonstrated the exponential growth of the solution space.

Finally, Q7 diagnosed the repeated subset evaluation in the Knapsack problem and introduced Dynamic Programming. The two implementations returned the same optimal value, while the redesigned method used fewer counted operations for the selected instance.

The overall lesson is that algorithm selection should consider input characteristics, computational complexity, memory requirements and actual experimental evidence. A redesign should not be accepted merely because it uses a more advanced technique; it should be validated against the original implementation using equivalent test conditions.